In [2]:
```
import os
import numpy as np
from tensorflow.keras.datasets import mnist

# Load dataset (auto-downloads first time)
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Create folder in current working directory
save_dir = "mnist_data"
os.makedirs(save_dir, exist_ok=True)

# Save arrays as .npy files
np.save(os.path.join(save_dir, "X_train.npy"), X_train)
np.save(os.path.join(save_dir, "y_train.npy"), y_train)
np.save(os.path.join(save_dir, "X_test.npy"), X_test)
np.save(os.path.join(save_dir, "y_test.npy"), y_test)

print("MNIST dataset saved in folder:", save_dir)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
MNIST dataset saved in folder: mnist_data
```

In [3]:
```
# Import libraries
import tensorflow as tf
from tensorflow.keras import layers, models

from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize pixel values (0-255 → 0-1)
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN (add channel dimension)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Build CNN model
model = models.Sequential()

model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Flatten())
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

# Compile model
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
model.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.1)

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)

print("\nTest Accuracy:", test_acc)

# Predict one sample
import numpy as np

sample = X_test[0].reshape(1, 28, 28, 1)
prediction = model.predict(sample)

print("Predicted Digit:", np.argmax(prediction))
```

```
Epoch 1/5
C:\Users\admin\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
844/844 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.9478 - loss: 0.1726 - val_accuracy: 0.9843 - val_loss: 0.0518
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.9833 - loss: 0.0533 - val_accuracy: 0.9903 - val_loss: 0.0385
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.9879 - loss: 0.0364 - val_accuracy: 0.9885 - val_loss: 0.0405
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9913 - loss: 0.0271 - val_accuracy: 0.9875 - val_loss: 0.0417
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 11s 13ms/step - accuracy: 0.9935 - loss: 0.0199 - val_accuracy: 0.9905 - val_loss: 0.0360
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9880 - loss: 0.0375

Test Accuracy: 0.9879999756813049
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step
Predicted Digit: 7
```

In [4]:

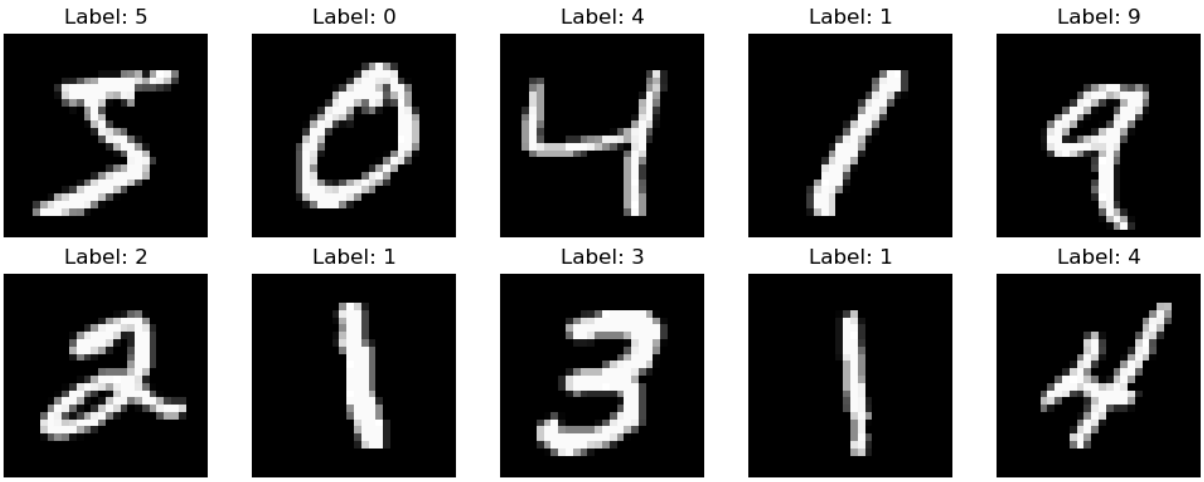
```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Display first 10 images
plt.figure(figsize=(10, 4))
```

```
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')

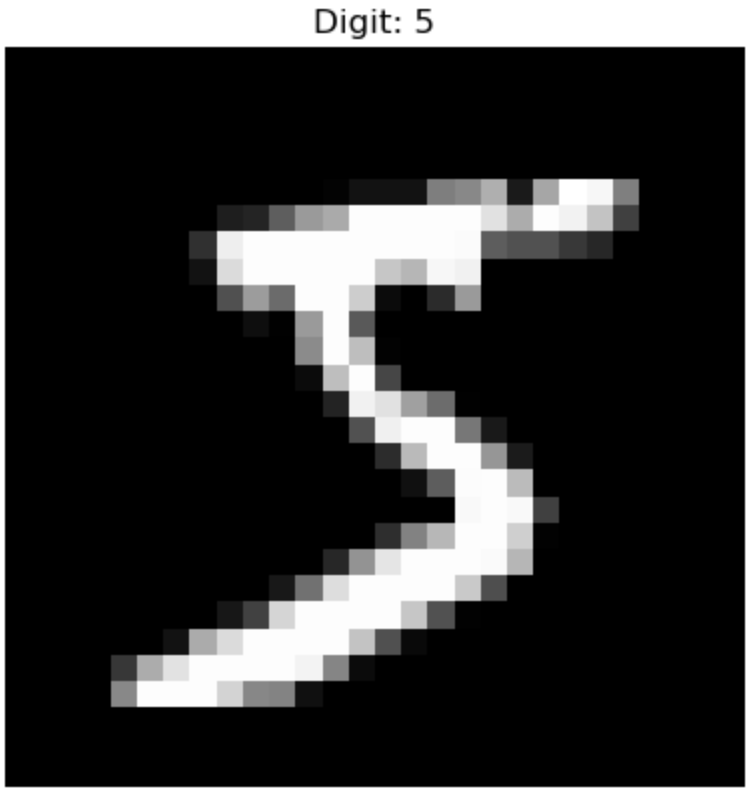
plt.tight_layout()
plt.show()
```



In [5]:
```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

(X_train, y_train), _ = mnist.load_data()

plt.imshow(X_train[0], cmap='gray')
plt.title(f"Digit: {y_train[0]}")
plt.axis('off')
plt.show()
```

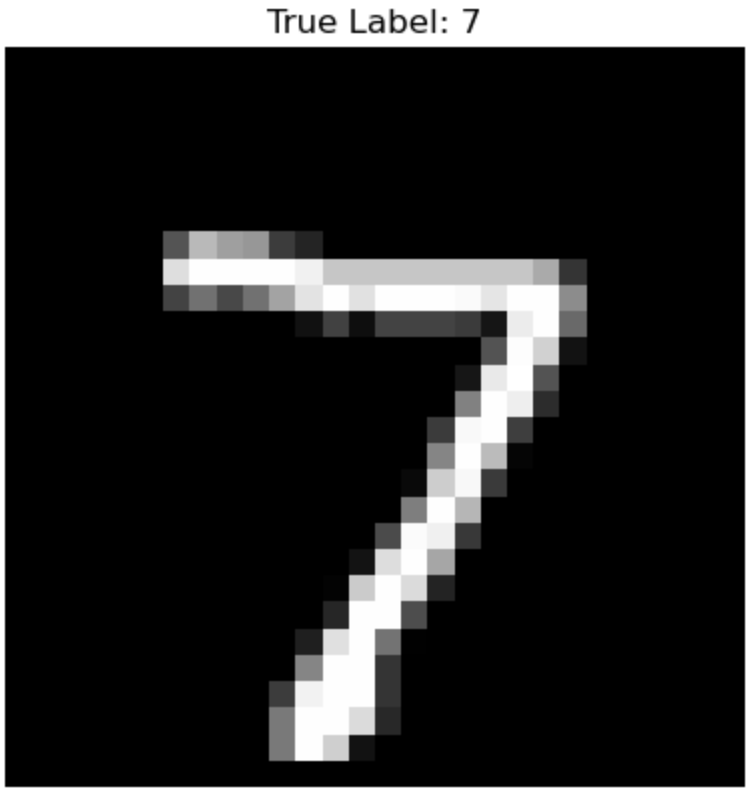


In [6]:
```
import numpy as np
import matplotlib.pyplot as plt

# pick a test image
img = X_test[0]

# display image
plt.imshow(img.reshape(28, 28), cmap='gray')
plt.title(f"True Label: {y_test[0]}")
plt.axis('off')
plt.show()

# prediction
prediction = model.predict(img.reshape(1, 28, 28, 1))
print("Predicted Digit:", np.argmax(prediction))
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
Predicted Digit: 7
```

In [7]:
```python
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import mnist
from tensorflow.keras import layers, models

from sklearn.metrics import confusion_matrix, classification_report

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Build CNN model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
```

```
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train model
model.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.1)

# Predict test data
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

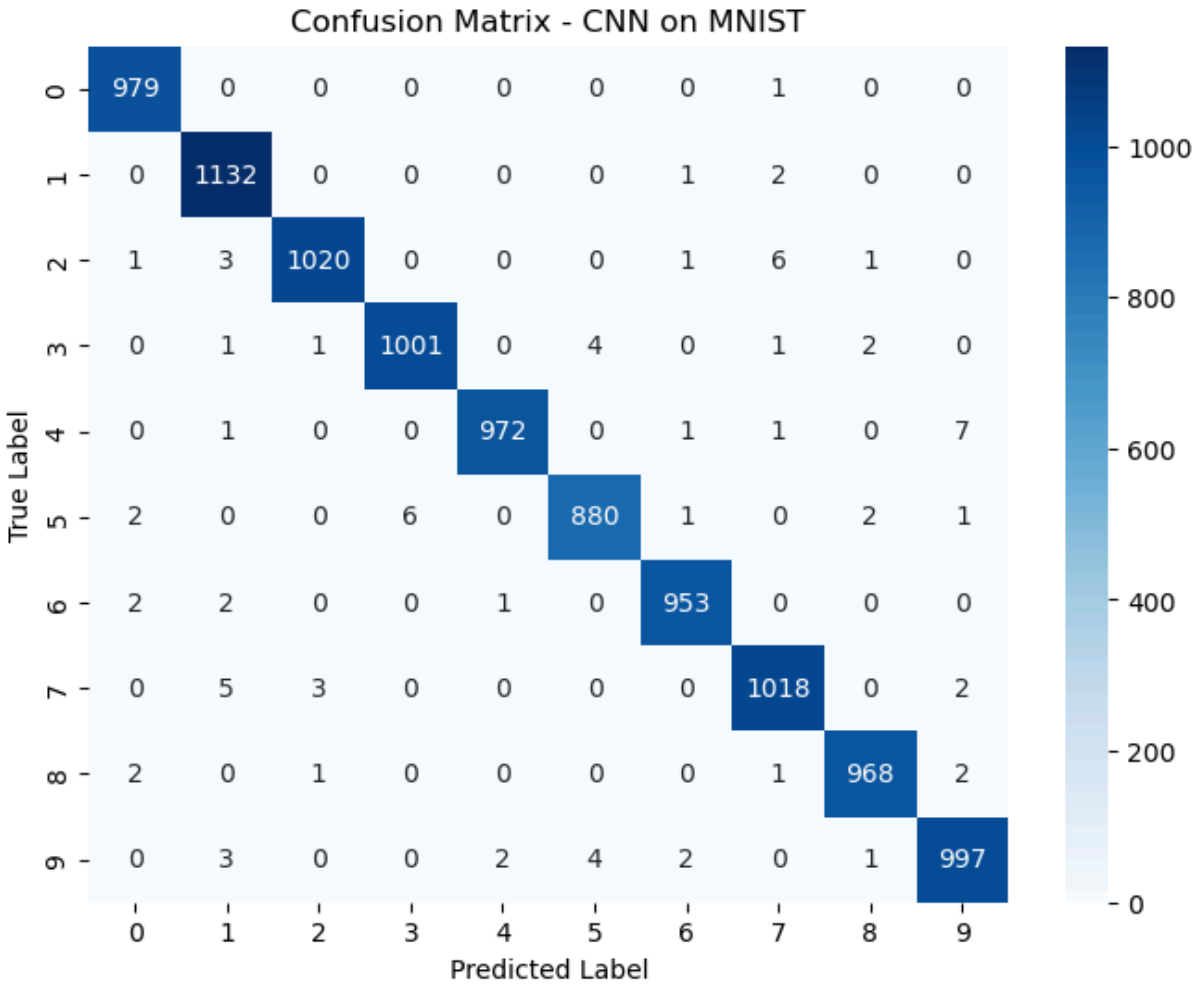
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix - CNN on MNIST")
plt.show()

# Classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Epoch 1/5
C:\Users\admin\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
844/844 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.9514 - loss: 0.1641 - val_accuracy: 0.9827 - val_loss: 0.0555
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.9849 - loss: 0.0486 - val_accuracy: 0.9873 - val_loss: 0.0419
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.9893 - loss: 0.0332 - val_accuracy: 0.9888 - val_loss: 0.0388
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9924 - loss: 0.0234 - val_accuracy: 0.9893 - val_loss: 0.0369
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 12s 14ms/step - accuracy: 0.9939 - loss: 0.0190 - val_accuracy: 0.9915 - val_loss: 0.0375
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	1.00	980
1	0.99	1.00	0.99	1135
2	1.00	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	1.00	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

In [ ]: